

LECTURE 4

Database Architecture and Data Independence

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

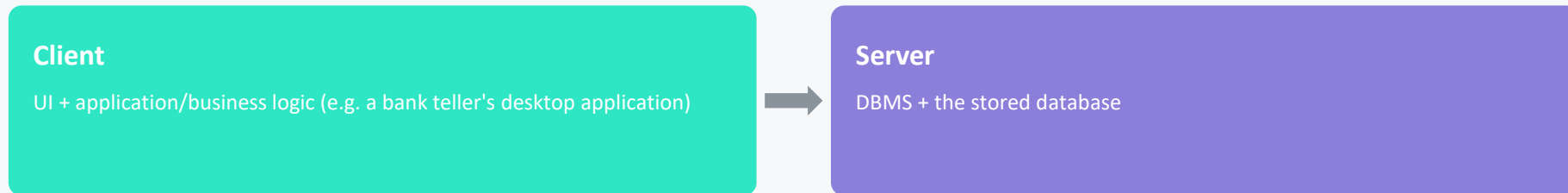
In This Lecture

- Two-tier client-server architecture, and its limitations
- Three-tier client-server architecture, and why it emerged
- The ANSI-SPARC three-schema architecture: external, conceptual, and internal levels
- External/conceptual and conceptual/internal mappings, and what each one does
- Logical vs. physical data independence — precisely defined, with worked examples

Two-Tier Client-Server Architecture

Before looking inside a DBMS, it's worth asking where it physically runs. In a two-tier architecture, the system splits into exactly two parts:

Two-tier client-server architecture



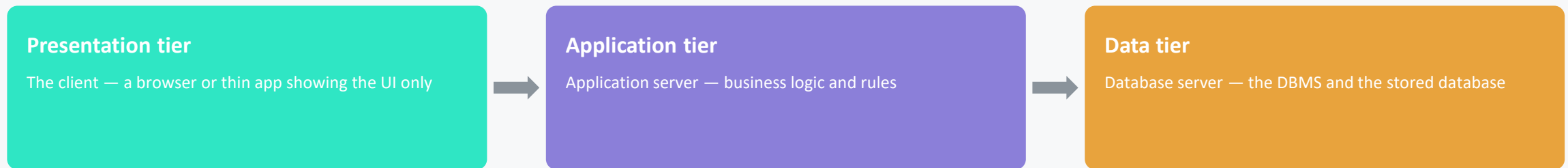
A classic example: an old-style desktop banking application on each teller's workstation, connecting straight to a central database server over the bank's internal network.

The limits of two tiers. This works well for a limited number of clients on a trusted internal network, but every client machine needs the full application installed and updated, business logic is duplicated across every client, and the database server must handle a direct connection from every single client — it stops scaling gracefully as users grow.

Three-Tier Client-Server Architecture

A three-tier architecture splits the client's responsibilities further, inserting a middle layer between the user-facing client and the database server:

Three-tier client-server architecture



A modern online banking website is a textbook three-tier system: the webpage is the presentation tier, the bank's application servers enforcing daily transfer limits are the application tier, and the account database is the data tier.

Client-server architecture is about physical location; the three-schema architecture next is about logical organization. A single-tier desktop database and a three-tier web application can both internally follow the same three-schema organization.

The ANSI-SPARC Three-Schema Architecture

The standard blueprint for how a DBMS organizes a database's structure into three levels, each describing the same underlying data from a different point of view:

The ANSI-SPARC three-schema architecture

External Level

Many external schemas, one per user group — each shows only the data that group needs

Conceptual Level

One conceptual schema — the complete community view of the whole database's structure and rules

Internal Level

One internal schema — how the data is actually stored on physical media

Three schemas, not three copies of the data. There is exactly one physical copy of the data, described at the internal level. The external and conceptual levels are descriptions — schemas — not additional data stores.

External Level

The **external level** is closest to individual users: a number of external schemas (user views), one per distinct user group, each describing only the part of the database that group cares about.

Worked example: a university enrollment database. The conceptual schema holds full Student, Course, Enrollment, and Instructor relations, including sensitive fields like Student.cnic and Instructor.salary.

Two external schemas over the same conceptual data

External schema: Course Advisor

StudentName, CourseTitle, Grade — no CNIC, no salary data at all

External schema: Payroll Clerk

InstructorName, Department, Salary — no student data at all

Neither view shows the whole conceptual schema, and neither is aware the other exists — each is tailored to exactly what its user group is authorized to see and needs to work with.

Conceptual Level and Internal Level



Conceptual Level

Exactly one conceptual schema: a single, complete, community-wide view of the entire database's logical structure — every entity, relationship, and constraint — described without reference to physical storage, and without favoring any one external view.

For the university example: the full set of relations (Student, Course, Enrollment, Instructor) with all attributes, primary and foreign keys, and business rules — independent of whether data ends up on a hard disk, an SSD, or split across several servers.



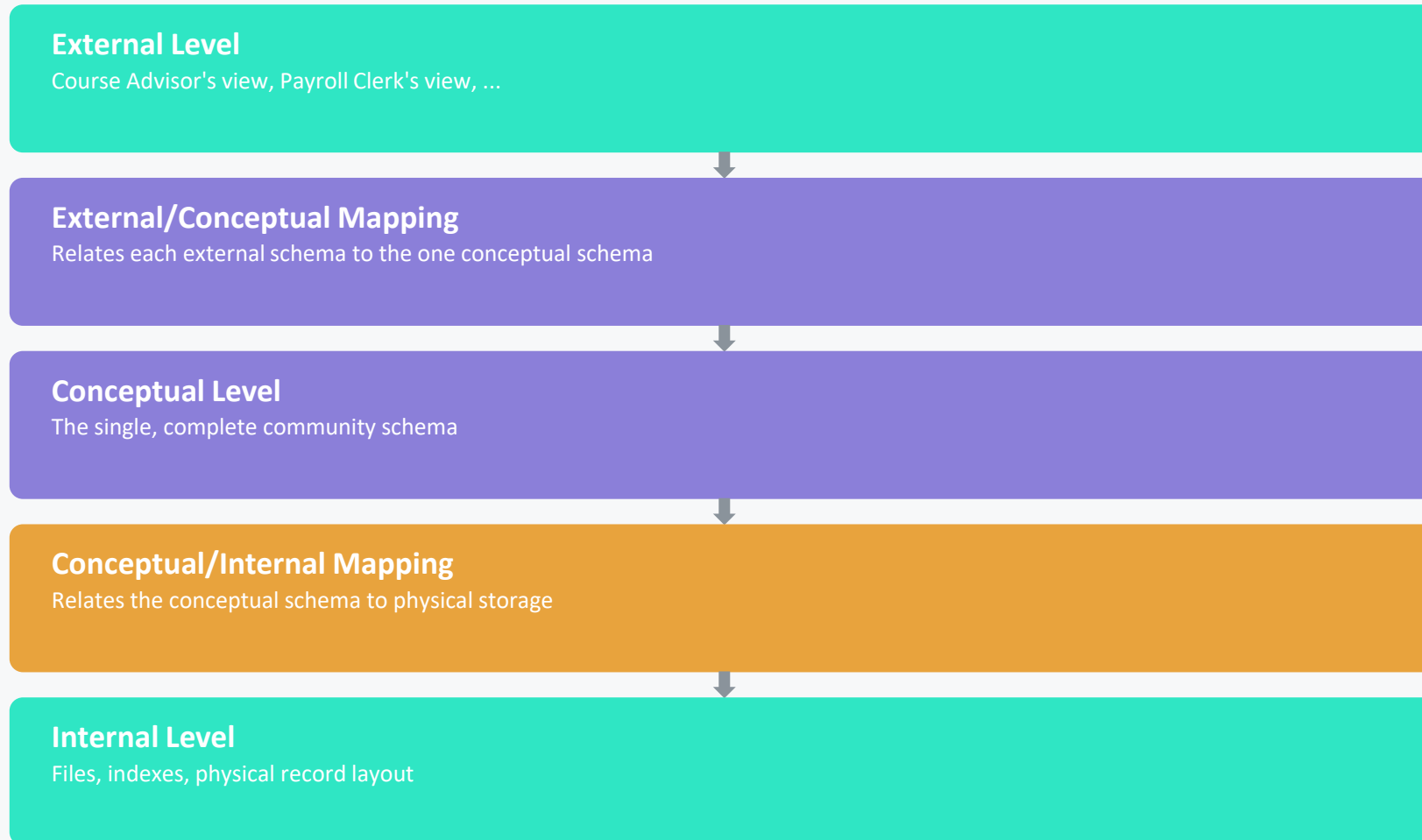
Internal Level

Exactly one internal schema, describing the physical storage representation: what files exist, what indexes exist, how records are laid out on disk, what compression or encoding is used. The only level that concerns itself with actual storage media.

For the university example: Student records stored in a B-tree-indexed file ordered by studentId, with a separate hash index on cnic — details no external-level user, and not even the conceptual schema, needs to know about.

Mappings Between the Levels

Because the three levels describe the same data differently, the DBMS needs a way to translate a request made at one level into an action at another:



Logical Data Independence

The capacity to change the conceptual schema without having to change existing external schemas or the application programs built against them.

It is achieved because a change to the conceptual schema only requires updating the external/conceptual mapping — the external schemas themselves stay exactly as they were.

Example. The university adds a new Instructor.officePhone attribute to the conceptual schema. The Course Advisor's external schema never referenced instructor data at all, so nothing about it needs to change, and every application built against it keeps working — completely unaware the conceptual schema grew a new column.

If the change is “we added a column” or “we split a relation in two,” think logical.

Physical Data Independence

The capacity to change the internal schema — how data is physically stored — without having to change the conceptual schema (and therefore not external schemas or applications either).

Example. The DBA adds a new index on Student.cnic, or migrates storage from spinning disks to SSDs, or reorganizes how Student records are laid out in their file. None of this touches the conceptual schema — only the conceptual/internal mapping changes.

Logical data independence

Conceptual schema changes -> only the external/conceptual mapping updates -> external schemas untouched

Physical data independence

Internal schema changes -> only the conceptual/internal mapping updates -> conceptual schema untouched

Anchor them by which schema changes: logical insulates users from conceptual-level changes; physical insulates the conceptual level from internal, physical-storage changes.

Key Takeaways

- Two-tier client-server architecture splits a system into client and database server; three-tier adds a middle application-server tier for business logic and scalability
- The ANSI-SPARC three-schema architecture organizes a database into external (many user views), conceptual (one complete community schema), and internal (physical storage) levels — three descriptions, not three copies of the data
- The external/conceptual mapping connects user views to the conceptual schema; the conceptual/internal mapping connects the conceptual schema to physical storage
- Logical data independence: changing the conceptual schema without forcing changes to external schemas or applications
- Physical data independence: changing the internal (physical storage) schema without forcing changes to the conceptual schema — and therefore not external schemas either